

Bài 22: Optional trong Java

Optional là một class trong package java.util dùng để biểu diễn một giá trị có thể tồn tại hoặc không tồn tại.

Optional giúp code thể hiện rõ khả năng thiếu dữ liệu và giảm nguy cơ NullPointerException khi xử lý các giá trị có thể là null.

1. Vấn đề với null

Khi một method có thể trả về null:

```
public String findNameById(int id) {  
    return null;  
}
```

Code sử dụng phải tự kiểm tra:

```
String name = findNameById(1);  
  
if (name != null) {  
    System.out.println(name.toUpperCase());  
}
```

Nếu quên kiểm tra:

```
String name = findNameById(1);  
System.out.println(name.toUpperCase()); // Có thể NullPointerException
```

Optional biểu diễn rõ hơn rằng kết quả có thể không tồn tại:

```
public Optional<String> findNameById(int id) {  
    return Optional.empty();  
}
```

2. Tạo Optional

Optional.empty()

Tạo một Optional không chứa giá trị:

```
Optional<String> emptyValue = Optional.empty();

System.out.println(emptyValue.isEmpty()); // true
```

Optional.of()

Dùng khi chắc chắn giá trị không null:

```
Optional<String> name = Optional.of("An");

System.out.println(name.get()); // An
```

Nếu truyền null cho of():

```
Optional<String> value = Optional.of(null); // NullPointerException
```

Optional.ofNullable()

Dùng khi giá trị có thể null:

```
String nameFromDatabase = null;

Optional<String> name =
    Optional.ofNullable(nameFromDatabase);

System.out.println(name.isEmpty()); // true
```

Bảng so sánh:

Cách tạo	Khi giá trị null
Optional.empty()	Tạo Optional rỗng
Optional.of(value)	Ném NullPointerException
Optional.ofNullable(value)	Tạo Optional rỗng

3. Kiểm tra Optional

isPresent()

Kiểm tra Optional có chứa giá trị hay không:

```
Optional<String> name = Optional.of("An");

if (name.isPresent()) {
    System.out.println(name.get());
}
```

isEmpty()

Kiểm tra Optional không chứa giá trị:

```
Optional<String> name = Optional.empty();

if (name.isEmpty()) {
    System.out.println("Không có tên");
}
```

isEmpty() được giới thiệu từ Java 11. Với Java 8, có thể dùng:

```
if (!name.isPresent()) {
    System.out.println("Không có tên");
}
```

4. Lấy giá trị bằng get()

get() trả về giá trị bên trong Optional:

```
Optional<String> name = Optional.of("An");

String result = name.get();
System.out.println(result);
```

Nếu Optional rỗng:

```
Optional<String> empty = Optional.empty();

empty.get(); // NoSuchElementException
```

Không nên gọi get() một cách tùy tiện. Nên dùng các method cung cấp giá trị mặc định hoặc xử lý an toàn hơn.

5. Giá trị mặc định với orElse()

orElse() trả về giá trị bên trong nếu có, nếu không thì trả về giá trị mặc định.

```
Optional<String> name = Optional.empty();

String result = name.orElse("Unknown");

System.out.println(result); // Unknown
```

Nếu có giá trị:

```
Optional<String> name = Optional.of("An");

String result = name.orElse("Unknown");

System.out.println(result); // An
```

6. orElse() luôn được đánh giá

Tham số của orElse() được tạo hoặc gọi ngay cả khi Optional đã có giá trị.

```
String result = optionalValue.orElse(createDefaultValue());
```

createDefaultValue() vẫn có thể được gọi dù optionalValue không rỗng.

7. Giá trị mặc định với orElseGet()

orElseGet() nhận một Supplier và chỉ gọi Supplier khi Optional rỗng.

```
Optional<String> name = Optional.empty();

String result = name.orElseGet(() -> createDefaultName());
```

Có thể viết:

```
String result = name.orElseGet(() -> "Unknown");
```

So sánh:

Method	Cách tính giá trị mặc định
orElse(value)	Luôn đánh giá value

Method	Cách tính giá trị mặc định
<code>orElseGet(supplier)</code>	Chỉ đánh giá khi Optional rỗng

Nên dùng `orElseGet()` khi việc tạo giá trị mặc định tốn chi phí.

8. Ném exception với `orElseThrow()`

Dùng `orElseThrow()` khi không có giá trị thì muốn báo lỗi.

```
Optional<String> name = Optional.empty();  
  
String result = name.orElseThrow();
```

Có thể cung cấp loại exception cụ thể:

```
String result = name.orElseThrow(  
    () -> new IllegalArgumentException("Không tìm thấy tên")  
);
```

Ví dụ trong service:

```
public User findUserById(int id) {  
    return userRepository.findById(id)  
        .orElseThrow(() ->  
            new UserNotFoundException("Không tìm thấy user " +  
id));  
}
```

9. `ifPresent()`

`ifPresent()` thực hiện một hành động nếu Optional có giá trị.

```
Optional<String> name = Optional.of("An");  
  
name.ifPresent(value -> System.out.println(value));
```

Có thể dùng method reference:

```
name.ifPresent(System.out::println);
```

Nếu Optional rỗng, Consumer không được gọi.

10. ifPresentOrElse()

ifPresentOrElse() xử lý cả hai trường hợp có và không có giá trị.

```
Optional<String> name = Optional.empty();

name.ifPresentOrElse(
    value -> System.out.println("Tên: " + value),
    () -> System.out.println("Không có tên")
);
```

ifPresentOrElse() được giới thiệu từ Java 9.

11. Lọc giá trị với filter()

filter() giữ lại giá trị nếu điều kiện đúng. Nếu điều kiện sai, kết quả là Optional rỗng.

```
Optional<Integer> age = Optional.of(20);

Optional<Integer> adultAge =
    age.filter(value -> value >= 18);

System.out.println(adultAge.isPresent()); // true
```

Ví dụ:

```
Optional<Integer> age = Optional.of(15);

Optional<Integer> adultAge =
    age.filter(value -> value >= 18);

System.out.println(adultAge.isEmpty()); // true
```

12. Biến đổi giá trị với map()

map() biến đổi giá trị bên trong Optional.

```
Optional<String> name = Optional.of("nguyen van an");

Optional<String> upperName =
    name.map(String::toUpperCase);

System.out.println(upperName.get()); // NGUYEN VAN AN
```

Nếu Optional ban đầu rỗng, map() không gọi function và kết quả vẫn là Optional rỗng.

Có thể nối nhiều thao tác:

```
String result = Optional.of(" Java ")
    .map(String::trim)
    .map(String::toUpperCase)
    .orElse("EMPTY");

System.out.println(result); // JAVA
```

13. Sự khác nhau giữa map() và flatMap()

Dùng map() khi function trả về một giá trị bình thường:

```
Optional<String> name = Optional.of("An");

Optional<Integer> length =
    name.map(String::length);
```

Nếu function đã trả về Optional, dùng flatMap() để tránh Optional lồng nhau:

```
Optional<User> user = findUser();

Optional<String> email = user.flatMap(User::getEmail);
```

Giả sử:

```
class User {
    private Optional<String> email;

    public Optional<String> getEmail() {
        return email;
    }
}
```

Nếu dùng map():

```
Optional<Optional<String>> wrong =
    user.map(User::getEmail);
```

Nếu dùng flatMap():

```
Optional<String> correct =  
    user.flatMap(User::getEmail);
```

14. Kết hợp nhiều Optional với or()

or() trả về Optional khác nếu Optional hiện tại rỗng.

```
Optional<String> result = findFromCache()  
    .or(() -> findFromDatabase())  
    .or(() -> findFromFile());
```

or() được giới thiệu từ Java 9.

15. Optional với Stream

Optional có thể chuyển thành Stream bằng stream().

```
List<Optional<String>> values = List.of(  
    Optional.of("Java"),  
    Optional.empty(),  
    Optional.of("SQL")  
);  
  
List<String> result = values.stream()  
    .flatMap(Optional::stream)  
    .toList();  
  
System.out.println(result); // [Java, SQL]
```

Optional.stream() được giới thiệu từ Java 9.

16. Optional với primitive

Để tránh boxing không cần thiết, Java có các class:

- OptionalInt.
- OptionalLong.
- OptionalDouble.

Ví dụ:

```
OptionalInt number = OptionalInt.of(10);  
  
if (number.isPresent()) {
```

```
System.out.println(number.getAsInt());  
}
```

Giá trị mặc định:

```
OptionalInt empty = OptionalInt.empty();  
  
System.out.println(empty.orElse(0)); // 0
```

Các method tương ứng:

```
OptionalInt    → getAsInt()  
OptionalLong   → getAsLong()  
OptionalDouble → getAsDouble()
```

17. Optional làm kiểu trả về

Optional thường phù hợp làm kiểu trả về của method khi kết quả có thể không tồn tại.

```
public Optional<User> findById(int id) {  
    User user = databaseFind(id);  
    return Optional.ofNullable(user);  
}
```

Code gọi method:

```
findById(10)  
    .map(User::getName)  
    .ifPresent(System.out::println);
```

Hoặc:

```
User user = findById(10)  
    .orElseThrow(() ->  
        new IllegalArgumentException("Không tìm thấy user"));
```

18. Không nên dùng Optional làm field

Thông thường không nên khai báo:

```
class User {  
    private Optional<String> name;  
}
```

Thay vào đó, field có thể là:

```
class User {  
    private String name;  
}
```

Và method getter có thể trả về Optional:

```
public Optional<String> getName() {  
    return Optional.ofNullable(name);  
}
```

Optional chủ yếu được thiết kế để làm kiểu trả về, không phải để thay thế mọi field có thể null.

19. Không nên dùng Optional làm parameter

Không nên viết:

```
void printName(Optional<String> name) {  
}
```

Thường nên nhận kiểu dữ liệu bình thường và xử lý rõ ràng:

```
void printName(String name) {  
    if (name != null) {  
        System.out.println(name);  
    }  
}
```

Hoặc thiết kế API để caller truyền giá trị bắt buộc nếu giá trị đó thực sự không được phép thiếu.

20. Optional không làm dữ liệu bên trong trở nên immutable

Optional chỉ bảo vệ việc thiếu dữ liệu, không làm object bên trong bất biến.

```
List<String> names = new ArrayList<>();  
Optional<List<String>> optionalNames = Optional.of(names);
```

```
names.add("An");

System.out.println(optionalNames.get()); // [An]
```

List bên trong vẫn có thể thay đổi.

21. Optional và null

Không nên trả về null thay cho Optional:

```
public Optional<String> findName() {
    return null; // Sai thiết kế
}
```

Nên trả về:

```
public Optional<String> findName() {
    return Optional.empty();
}
```

Nếu method đã khai báo trả về Optional, nó nên luôn trả về một Optional object, có thể chứa giá trị hoặc rỗng.

22. Ví dụ đầy đủ

```
import java.util.Optional;

public class UserService {
    public Optional<String> findEmail(String username) {
        if ("an".equals(username)) {
            return Optional.of("an@example.com");
        }

        return Optional.empty();
    }

    public static void main(String[] args) {
        UserService service = new UserService();

        String email = service.findEmail("an")
            .filter(value -> value.contains("@"))
            .map(String::toLowerCase)
            .orElse("unknown@example.com");

        System.out.println(email);
    }
}
```

```
}  
}
```

23. Điểm chính

- Optional biểu diễn một giá trị có thể tồn tại hoặc không tồn tại.
- Dùng `Optional.empty()` để tạo Optional rỗng.
- Dùng `Optional.of()` khi chắc chắn giá trị không null.
- Dùng `Optional.ofNullable()` khi giá trị có thể null.
- Tránh gọi `get()` nếu chưa chắc Optional có giá trị.
- Dùng `orElse()`, `orElseGet()` hoặc `orElseThrow()` để lấy kết quả an toàn.
- Dùng `map()` để biến đổi giá trị.
- Dùng `flatMap()` khi function trả về Optional.
- Dùng `filter()` để giữ giá trị theo điều kiện.
- Optional thường phù hợp nhất làm kiểu trả về.

24. Thực hành tốt

- Dùng Optional để biểu diễn kết quả có thể thiếu.
- Không trả về null từ method có kiểu trả về Optional.
- Không dùng Optional chỉ để thay thế mọi biến null.
- Không gọi `get()` mà không kiểm tra hoặc xử lý trường hợp rỗng.
- Dùng `orElseGet()` nếu giá trị mặc định cần tính toán tốn chi phí.
- Dùng `orElseThrow()` khi thiếu dữ liệu là lỗi nghiệp vụ.
- Không dùng Optional làm field hoặc parameter nếu không thật sự cần.
- Chọn `OptionalInt`, `OptionalLong` hoặc `OptionalDouble` khi làm việc với primitive.

25. Các lỗi thường gặp

Lỗi	Nguyên nhân
<code>NoSuchElementException</code>	Gọi <code>get()</code> trên Optional rỗng
<code>NullPointerException</code> khi tạo Optional	Dùng <code>Optional.of(null)</code>
Optional bị null	Method trả về null thay vì <code>Optional.empty()</code>
Tạo Optional lồng nhau	Dùng <code>map()</code> khi function đã trả về Optional
Tính giá trị mặc định không cần thiết	Dùng <code>orElse()</code> thay vì <code>orElseGet()</code>
Dùng Optional quá mức	Dùng Optional cho mọi field, parameter hoặc biến local
Nhầm Optional với object bất biến	Object bên trong Optional vẫn có thể thay đổi

26. Ghi nhớ

`Optional<T>` = có thể chứa một giá trị kiểu T hoặc không chứa gì

```
of(value)           → value chắc chắn không null  
ofNullable(value)   → value có thể null  
empty()             → không có giá trị
```

```
map()               → biến đổi giá trị  
flatMap()          → biến đổi sang Optional khác  
filter()           → giữ giá trị nếu điều kiện đúng  
orElse()            → giá trị mặc định  
orElseGet()         → tạo giá trị mặc định khi cần  
orElseThrow()       → ném exception nếu rỗng
```